PRODUCT REQUIREMENTS DOCUMENT

DiskyDisk

---

1. 🎯 Product Overview

1.1 Problem

Users don't know:

- what is taking up disk space
- where large files are hidden
- what is safe to delete

System tools are:

- slow
- non-visual
- not actionable

1.2 Solution

A **high-performance disk analyzer** that:

- scans storage in seconds
- visualizes space usage (sunburst map)
- identifies large/unnecessary files
- enables safe cleanup

---

2. 👤 Target Users

Primary

- developers (huge caches, node_modules, docker)
- students (downloads, random files)
- general laptop users (low storage issues)

Secondary

- power users (SSD optimization)
- content creators (large media files)

---

3. 🚀 Goals

MVP Goals

- scan disk quickly (<10 sec for ~100GB)
- visualize storage clearly
- identify largest files/folders
- allow safe deletion

Success Metrics

- scan time < 10s (SSD)
- UI response < 100ms
- crash rate < 1%
- user finds >1GB removable data within 1 min

---

4. 🧩 Core Features

4.1 Disk Scanning Engine

- recursive traversal
- size aggregation
- metadata extraction
- parallel scanning

4.2 Visualization

- sunburst (radial treemap)
- zoom into folders
- hover info
- color-coded segments

4.3 File Insights

- largest files list
- breakdown by:
    - file type
    - folder
- % of total disk

4.4 File Actions

- reveal in file explorer
- move to trash
- ignore folder
- rescan

4.5 Search & Filter

- search by name
- filter by:
    - size
    - type
    - date

---

5. 🧠 Advanced Features (Post-MVP)

- duplicate file detection
- unused file detection (last accessed)
- dev junk cleaner:
    - node_modules
    - Docker
    - build folders
- scan history + comparison
- AI cleanup suggestions

---

6. 🏗️ SYSTEM ARCHITECTURE

High-Level Architecture

```
+----------------------+
|   React Frontend     |
| (UI + Visualization)|
```

```
+----------+-----------+
        |
        | IPC (Tauri bridge)
        |
+----------v-----------+
|    Rust Backend     |
| (Scanner Engine)    |
+----------+-----------+
        |
        |
+----------v-----------+
|  File System (OS)   |
+----------------------+
```

---

7. 🧱 COMPONENT BREAKDOWN

7.1 Frontend (React + TS)

Responsibilities

- render UI
- display visualization
- handle user interactions
- call backend APIs

Key Components

```
App
├── Sidebar
├── SunburstChart
├── FileList
├── Breadcrumbs
├── SearchBar
└── ActionPanel
```

7.2 Backend (Rust via Tauri)

Responsibilities

- scan file system
- process tree structure
- handle file operations
- manage caching

---

8. ⚙️ DATA FLOW

Scan Flow

User clicks "Scan"
↓
Frontend → Tauri command
↓
Rust scanner starts
↓
Parallel traversal
↓
Tree structure built
↓
JSON returned
↓
Frontend renders visualization

---

9. 📦 DATA MODELS

9.1 File Node

```
{
 "name": "Downloads",
 "path": "/Users/ayush/Downloads",
 "size": 123456789,
 "type": "directory",
 "children": [...]
}
```

9.2 File Metadata

```
{
 "path": "...",
 "size": 12345,
 "extension": "mp4",
 "lastModified": 1678900000,
 "isDirectory": false
}
```

9.3 Scan Result

```
{
 "root": Node,
 "totalSize": 512000000000,
 "largestFiles": [...],
 "fileTypeBreakdown": {...}
}
```

---

10. ⚡ PERFORMANCE DESIGN

Techniques

Parallel Traversal

- Rust rayon threads
- multi-core utilization

Lazy Rendering

- don’t render entire tree
- only visible segments

Incremental Updates (future)

- stream partial scan results

Caching

- store previous scan
- diff only changed folders

---

11. 🧪 EDGE CASE HANDLING

11.1 Permissions

- skip restricted folders
- log errors
- show warnings in UI

11.2 Symlinks

- skip initially
- avoid infinite loops

11.3 Huge Directories

- chunk processing
- limit recursion depth initially

11.4 Corrupted Files

- skip safely

---

12. 🔐 SECURITY & SAFETY

- no permanent delete (only trash)
- confirmation dialogs
- sandboxed file operations
- no network calls (privacy-first)

---

13. 🎨 UX DESIGN

Key UX Principles

- instant feedback
- smooth zoom animations
- minimal clicks to action
- clear size representation

Visual Elements

- sunburst = main focus
- sidebar = controls
- breadcrumbs = navigation
- tooltip = details

---

14. 🧰 TECH STACK

Frontend

- React + TypeScript
- D3.js (visualization)
- TailwindCSS

Backend

- Rust
- Tauri

Libraries

- walkdir (filesystem traversal)
- rayon (parallelism)
- serde (JSON)

---

15. 🗂️ PROJECT STRUCTURE

```
root/
├── src-tauri/
│   ├── scanner.rs
│   ├── tree.rs
│   ├── commands.rs
│   └── main.rs
│
├── frontend/
│   ├── components/
│   │   ├── Sunburst.tsx
│   │   ├── FileList.tsx
│   │   └── Sidebar.tsx
│   ├── pages/
│   └── utils/
│
└── package.json
```

---

16. 🛠️ API DESIGN (TAURI COMMANDS)

Scan

invoke("scan_disk", { path: "/" })

Delete File

invoke("move_to_trash", { path: "/file" })

Reveal

invoke("reveal_in_explorer", { path: "/file" })

---

17. 📊 ALGORITHMS

Size Aggregation

Bottom-up:

folder size = sum(children sizes)

Largest Files

- maintain max heap
- or sort after scan

---

18. 📅 DEVELOPMENT ROADMAP

Phase 1 (Week 1)

- setup Tauri + React
- basic scanner
- return JSON

Phase 2 (Week 2)

- visualization
- zoom + hover
- largest files list

Phase 3 (Week 3)

- actions (delete, reveal)
- filters/search

Phase 4 (Week 4)

- performance tuning
- caching
- polish UI

---

19. ⚠️ RISKS

Risk	Solution
Slow scan	Rust + parallel
UI lag	lazy rendering
permissions issues	graceful fallback
huge data	streaming

---

20. 💰 FUTURE MONETIZATION

- pro features:
  - duplicate finder
  - smart cleanup
  - cloud scan
- enterprise:
  - disk analytics dashboard

---

21. 🔥 DIFFERENTIATION IDEAS

This is where you win:

Option 1: Dev Cleaner

- detect dev junk automatically

Option 2: AI Cleanup

- suggest deletions

Option 3: Storage Timeline

- track growth over time

---

22. 🧪 TESTING STRATEGY

- unit tests (Rust scanner)
- UI snapshot tests
- stress test:
  - millions of files
- permission edge tests

---

23. 📦 DEPLOYMENT

- build via Tauri
- release:
  - macOS
  - Windows
- auto-update (later)

---

24. FINAL TAKE

This is a **very strong project** if you execute it right:

- systems + UI + performance
- real-world usefulness
- easy to demo
- monetizable